

AI-RAG Week 3 — Lab Guide

Self-paced lab — ~2.5 hours total, split across two evenings.

Day	Lab time	Steps
Day 1 evening	45 min	Step 1 (8 sub-steps)
Day 2 evening	90 min	Steps 2, 3, 4 (12 sub-steps)

About this guide. Both instructor and learner follow this guide. The instructor walks through it on screen during live sessions; the learner walks through it at their own pace in the lab. **Code lives in pre-uploaded reference files** in your lab environment — see *Reference files* below for the inventory. This guide describes what to do, what to observe, and what it means; the reference files show the exact code.

Table of contents

1. [Before you begin](#)
 2. [Reference files in your environment](#)
 3. [Step 1 — Build the FastAPI service + Streamlit UI](#)
 4. [Step 2 — Mocked unit tests](#)
 5. [Step 3 — Stress-test the API](#)
 6. [Step 4 — Update ADR with the API contract](#)
 7. [End-of-week summary](#)
 8. [Troubleshooting reference](#)
-

Before you begin

What you should already have (from W1 + W2)

- A working `src/pipeline/` package with `pipeline.py` (exposing `ask_llm`, `ask_llm_with_retry`, `Question`, `Answer`), `fake_llm.py`, `settings.py` (`Settings` + `RunSummary`), `logging_config.py`, `store.py`, `query_results.py`.
- W2's async batch pipeline tested against real questions; SQLite `results.db` populated.
- `docs/adr/0001-capstone-framing.md` from W1.
- An OpenAI API key with a few dollars of credit.

If any of these is missing, finish the W2 lab before proceeding.

Install the W3 dependencies

From your capstone repo root:

```
pip install -r requirements.txt
```

The W3 additions: `fastapi`, `uvicorn[standard]`, `streamlit`, `requests`, `pytest`, `pytest-asyncio`, `httpx`.

Vocareum notes

- FastAPI listens on port **8000**; Streamlit on port **8501**. Both are reachable through Vocareum's port-forwarding URLs (look for "Open in new tab" links in the Vocareum terminal pane).
- Export your API key before starting uvicorn:

```
export OPENAI_API_KEY=sk-...
```

Two model layers (important)

W3 introduces a deliberate split between the **public API contract** and the **W2 internal models**:

Layer	Lives in	Question field	Answer fields
Public API (W3) — what HTTP clients send/receive	<code>api/main.py</code>	<code>question</code>	<code>content</code> , <code>cost_usd</code> , <code>retries</code>
Internal pipeline (W2) — what <code>ask_llm</code> operates on	<code>src/pipeline/pipeline.py</code>	<code>text</code>	<code>question</code> , <code>text</code> , <code>cost_usd</code> , <code>retries</code>

The W3 ADR locks the public contract at `question/content`. The W2 pipeline uses `text` everywhere internally. **The translation happens inside each endpoint handler** in `api/main.py` — every endpoint takes a public `Question`, builds an internal `_PipelineQuestion(text=q.question)`, calls `ask_llm`, then returns a public `Answer(content=pipeline_ans.text, ...)`.

This separation means W4+ can swap the W2 internals without touching the W3 contract. It's a small piece of bookkeeping today; it's the reason the contract holds for the next 27 weeks.

Reference files in your environment

All of these are **pre-uploaded** into your lab environment. This guide tells you when to consult each.

Path	What it is
<code>api/main_starter.py</code>	Skeleton with <code>TODO 1b</code> → <code>TODO 1f</code> markers. Copy to <code>api/main.py</code> to start work.
<code>api/main_reference.py</code>	Completed <code>api/main.py</code> . The canonical

Path	What it is
	answer; consult when stuck.
ui/app_streamlit.py	Completed Streamlit UI.
tests/conftest.py	Pytest path setup. Use as-is.
tests/test_pipeline_reference.py	Completed pipeline tests (2 tests). Reference for sub-steps 2b–2c.
tests/test_api_reference.py	Completed API tests (2 tests). Reference for sub-step 2b.
pytest.ini	Pytest config. Use as-is.
scripts/stress_test.py	Stress harness. Use as-is in sub-step 3d.
docs/adr/0002-api-contract-template.md	ADR template. Copy to docs/adr/0002-api-contract.md and fill in.
requirements.txt	W3 dependencies merged into your existing.

Workflow rule. Edit the *working* file (api/main.py, tests/test_pipeline.py, ...). Read the *reference* file when you need to check the exact form. Typing the shape yourself is part of how it lands.

Step 1 — Build the FastAPI service + Streamlit UI (45 min)

Outcome. By the end of Step 1, uvicorn is running your /ask (streaming) and /health endpoints; a Streamlit UI at port 8501 lets you ask 3 capstone-style questions and watch the answer stream in.

Step 1 at a glance

Sub-step	Time	What you build
1a	5 min	Project structure + deps
1b	5 min	FastAPI app instance
1c	5 min	/ask_batched (non-streaming reference)
1d	3 min	/health
1e	4 min	Smoke test via curl + /docs
1f	8 min	Streaming /ask
1g	10 min	Streamlit UI
1h	5 min	End-to-end smoke test

1a — Project structure & dependencies (5 min)

1. What we're doing & why. We need places for the API code (`api/`), the UI (`ui/`), and the tests (`tests/`). Plus five new pip packages before any of the rest works.

2. Where we are now. Your repo has `src/pipeline/` (from W2), `docs/`, `data/`, and your W2 scripts. There is no `api/`, no `ui/`, no `tests/`.

3. What we're about to change. Three new directories. Five new pip packages.

4. Make the change. From your repo root:

```
mkdir -p api ui tests scripts docs/adr
touch api/__init__.py ui/__init__.py tests/__init__.py
pip install -r requirements.txt
```

The `__init__.py` files turn the directories into proper Python packages so imports work cleanly.

5. Run it.

```
pip list | grep -E "fastapi|uvicorn|streamlit|pytest"
```

6. What you should see.

fastapi	0.110.0	← installed
pytest	7.4.3	← installed
pytest-asyncio	0.21.1	← installed
streamlit	1.32.0	← installed
uvicorn	0.27.0	← installed

7. What just happened. Directory shells in place. Five new packages installed. Nothing wired up yet — that's the next seven sub-steps.

Watch for. If `pip install` fails with `ResolutionImpossible`, check you haven't pinned an old `httpx` somewhere — `fastapi` and `pytest` both want `httpx>=0.26`.

1b — Create `api/main.py` with the FastAPI app instance (5 min)

1. What we're doing & why. Copy the starter file in and fill in the FastAPI app instance. Once this one line is wired, we can run `uvicorn` against an empty app — a useful sanity check before adding endpoints.

2. Where we are now. `api/` has only `__init__.py`. The starter is at `api/main_starter.py`.

3. What we're about to change. Copy the starter to `api/main.py`. Edit one line.

4. Make the change. Copy the starter into a working file:

```
cp api/main_starter.py api/main.py
```

Open `api/main.py` and read through the top of the file first — the starter already contains:

- Imports from `src.pipeline.pipeline` (the W2 engine): `ask_llm` as `_pipeline_ask_llm` and `Question` as `_PipelineQuestion`.

- The public `W3 Question` and `Answer` Pydantic models — these are the contract HTTP clients see.

You don't edit those. They're scaffolding for the endpoints you're about to add.

Find the placeholder line marked `# TODO 1b` near the top — it sets `app = None`. Replace that line with a real FastAPI app instance. Give it:

- a **title** — e.g. "Capstone API". Appears on `/docs`.
- a short **description** — what this service does. Stakeholders read this.
- a **version** — "1.0.0" is fine.

Completed shape: `api/main_reference.py`.

5. Run it.

```
uvicorn api.main:app --reload --port 8000
```

6. What you should see.

```
INFO: Will watch for changes in these directories: ['/your/path/capstone']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit) ←
INFO: Started reloader process [12453] using StatReload
INFO: Started server process [12455]
INFO: Application startup complete. ←
```

Open `http://localhost:8000/docs` in a browser. The Swagger UI page appears with no endpoints yet. That's expected.

7. What just happened. A live FastAPI server is listening on port 8000. It has no useful endpoints, but the framework is alive. **Keep uvicorn running** in this terminal for the next several sub-steps — `--reload` means it restarts automatically as you save changes to `api/main.py`.

Watch for. `ModuleNotFoundError: src.pipeline` → you're running uvicorn from the wrong directory. Run it from your repo root, with `src/`, `api/`, etc. as direct children.

1c — Add `/ask_batched` (non-streaming reference endpoint) (5 min)

1. What we're doing & why. A non-streaming endpoint is the simplest possible wiring around `ask_llm`. Build it first because it's easy to test with curl, and keep it permanently as the reference / fallback endpoint that returns the full `Answer` body in one shot.

2. Where we are now. `app` exists. No routes are wired.

3. What we're about to change. Add a `POST /ask_batched` endpoint that takes a `Question`, calls `ask_llm`, returns an `Answer`.

4. Make the change. In `api/main.py`, find the `# TODO 1c` block. Add an async function decorated with `@app.post("/ask_batched", response_model=Answer)`:

- Parameter: `q: Question` (the `W3` public model — has `q.question`).

- Return type: `Answer` (the W3 public model — has `content`, `cost_usd`, `retries`).
- Body:
 1. Translate the public input to the pipeline's internal model: `pipeline_q = _PipelineQuestion(text=q.question)`.
 2. Call the W2 pipeline: `pipeline_ans = await _pipeline_ask_llm(pipeline_q)`.
 3. Translate the internal answer to the public response: `return Answer(content=pipeline_ans.text, cost_usd=pipeline_ans.cost_usd, retries=pipeline_ans.retries)`.

The `response_model=Answer` parameter makes FastAPI validate your response against the `Answer` schema and document it on `/docs`. The three-line translation block is the W3-vs-W2 boundary in action — it'll repeat in every endpoint that calls `ask_llm`. Reference: `api/main_reference.py`.

Save. Uvicorn hot-reloads.

5. Run it. In a second terminal:

```
curl -X POST http://localhost:8000/ask_batched \
  -H "Content-Type: application/json" \
  -d '{"question": "What is the leave policy?"}'
```

6. What you should see.

```
{
  "content": "Employees are entitled to 24 days of paid leave per year ...", ←
  "cost_usd": 0.0001,
  "retries": 0
} ←
W2 placeholder
```

Takes 2–5 seconds — the LLM call runs synchronously. Streaming comes in 1f.

7. What just happened. Your W2 pipeline is now reachable over HTTP. Anyone with `curl` can ask it questions.

Watch for. If you get a 500, check the uvicorn terminal — the traceback is there. Most common cause: `OPENAI_API_KEY` env var not set in the shell you started uvicorn from.

1d — Add `/health` (3 min)

1. What we're doing & why. A liveness endpoint. Two lines of code. You'll be grateful for it the first time you deploy a service to Kubernetes (W29).

2. Where we are now. `/ask_batched` works. No `/health`.

3. What we're about to change. Add a GET `/health` endpoint returning `{"status": "ok"}`.

4. Make the change. In `api/main.py`, find the `# TODO 1d` block. Add an `async` function decorated with `@app.get("/health")` that returns the dictionary `{"status": "ok"}`. Save. Reference: `api/main_reference.py`.

5. Run it.

```
curl http://localhost:8000/health
```

6. What you should see.

```
{"status": "ok"}
```

7. What just happened. You have a probe endpoint. In W29 the container orchestrator will hit `/health` every 10 seconds to know whether to restart your service.

1e — Smoke test via curl + /docs (4 min)

1. What we're doing & why. Before adding streaming complexity, confirm both endpoints work and the auto-generated docs page is sensible.

2. Where we are now. `/ask_batched` and `/health` are wired.

3. What we're about to change. Nothing in the code. Just exercise it.

4. Make the change. Three checks:

```
# Check 1 — health
curl http://localhost:8000/health

# Check 2 — ask_batched with a real capstone question
curl -X POST http://localhost:8000/ask_batched \
  -H "Content-Type: application/json" \
  -d '{"question": "How does the procurement approval process work?"}'

# Check 3 — malformed request (no question field)
curl -X POST http://localhost:8000/ask_batched \
  -H "Content-Type: application/json" \
  -d '{}'
```

Then in a browser:

```
http://localhost:8000/docs
```

5. Run it. Run each check.

6. What you should see. A Swagger UI page listing `POST /ask_batched` and `GET /health`, with the `Question` and `Answer` schemas expanded. Click *Try it out* on `/ask_batched`, paste a question, hit `Execute`.

For check 3 you should see HTTP 422 with a body containing a `detail` list mentioning the `question` field is required:

```
{
  "detail": [{"type": "missing", "loc": ["body", "question"], "msg": "Field required", ...}]
}
```

7. What just happened. You confirmed: (1) the service is alive, (2) the round-trip to OpenAI works through your code, (3) Pydantic validation is doing its job. Baseline before adding streaming.

Watch for. If `/docs` is blank or shows a CSP error, a browser extension is interfering. Try incognito. On Vocareum, `/docs` works through the port-forwarded URL the same as any other route.

1f — Convert `/ask` to streaming (8 min)

1. What we're doing & why. The user-facing endpoint should stream. Tokens arriving over a few seconds feel alive; a 5-second pause then a wall of text feels broken. We add `/ask` (streaming) as a new endpoint, keeping `/ask_batched` as the non-streaming reference.

2. Where we are now. `/ask_batched` returns the full Answer in one shot. No streaming.

3. What we're about to change. Add two pieces to `api/main.py`:

1. An **async generator function** that takes a question and yields the answer in chunks.
2. A new `POST /ask` endpoint that returns a `StreamingResponse` wrapping that generator.

4. Make the change. In `api/main.py`, find the `# TODO 1f` block. Add:

1. An async generator `stream_answer(question: str)` that:
 - Awaits `ask_llm(question)` to get the full Answer.
 - Iterates over `answer.content.split(" ")` and yields each word with a trailing space.
 - Awaits `asyncio.sleep(0.05)` between yields — the 50 ms gap simulates inter-token spacing and makes streaming visible.
2. A `@app.post("/ask")` endpoint that takes `q: Question` and returns `StreamingResponse(stream_answer(q.question), media_type="text/plain")`.

Two rules worth re-reading:

- The generator function uses `async def` and `yield`, not `return`. That makes it an async generator — FastAPI iterates over it and writes each yielded chunk to the response stream.
- In W4 we'll stream from the LLM itself; for now the simulated 50 ms gap is what makes streaming visible.

Reference: `api/main_reference.py`.

5. Run it.

```
curl -N -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"question": "What is the leave policy?"}'
```

The `-N` flag is critical — it disables curl's output buffering so you actually see chunks arrive.

6. What you should see. Words appearing one at a time, ~50 ms apart:

```
Employees ← appears
get        ← 50 ms later
24         ← 50 ms later
days
of
paid
leave
per
year.
word answer ← total ~3-5s for a 60-
```

If you drop the `-N` you'll see the whole answer dump at once — that's curl buffering, not the server. The server is still streaming.

7. What just happened. You have a streaming `/ask` endpoint. The contract on it is the same as `/ask_batched` (request: `Question`) — only the response shape differs (`text/plain` stream vs `JSON Answer`). Both endpoints share the underlying `ask_llm` call.

Watch for.

- **Forgetting `await`** in front of `_pipeline_ask_llm(pipeline_q)`. Symptom: streaming endpoint hangs then 500s. `pipeline_ans` is a coroutine, not an `Answer`; `.text.split(" ")` raises `AttributeError`.
- **return instead of `yield`** turns the function into a coroutine that returns a list — FastAPI sends everything at once, no streaming.
- **Forgetting `media_type="text/plain"`**. FastAPI may infer the wrong type and clients may treat the stream as a single binary blob.

1g — Run the Streamlit UI (10 min)

1. What we're doing & why. A small UI that takes a text input, hits `/ask` with `stream=True`, and updates the displayed answer as chunks arrive. Stakeholders can use it. The W5 eval harness will use this same `/ask` endpoint.

2. Where we are now. Your API streams. No UI yet.

3. What we're about to change. Use the pre-uploaded Streamlit UI at `ui/app_streamlit.py`. Either run it directly, or copy it to `ui/app.py` first.

4. Make the change.

```
cp ui/app_streamlit.py ui/app.py
```

Open `ui/app_streamlit.py` and read it line by line — it's under 25 lines and worth understanding before running. The streaming mechanism on the client side has two pieces:

- `stream=True` on `requests.post(...)` opens the connection in streaming mode.

- `for chunk in response.iter_content(decode_unicode=True)` iterates chunks as they arrive.
- `st.empty()` returns a placeholder we can rewrite on each chunk; calling `.markdown(answer)` on the placeholder redraws the same spot with the accumulated answer. That's what makes the UI appear to type itself.

5. Run it. Keep uvicorn running in the first terminal. In a third terminal:

```
streamlit run ui/app.py --server.port 8501
or try this
python -m streamlit run ui/app.py --server.port 8501
```

6. What you should see.

```
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://192.168.x.x:8501
```

A browser tab opens (or open `localhost:8501` yourself). You see a "Capstone Q&A" page with a text input.

7. What just happened. You have a UI hitting your API. Type a question, click Ask, watch the words appear one at a time. Anyone — your sponsor, a peer — can try the capstone now without running Python.

Watch for.

- **UI sits at "Running..." forever.** Either the API isn't running, or it's on a different port than the UI expects. Check the uvicorn terminal; confirm `API_URL` inside `ui/app.py` matches the port uvicorn is listening on.
- **"Cannot reach the API" toast.** Same diagnosis. Check uvicorn.
- **Old Streamlit version.** If the UI doesn't update mid-stream, bump Streamlit to ≥ 1.32 .

1h — End-to-end smoke test with 3 capstone questions (5 min)

1. What we're doing & why. A final reality check: three real questions from your capstone domain, asked through the UI, all stream cleanly. If this works, Step 1 is complete.

2. Where we are now. API and UI both running.

3. What we're about to change. Nothing. Just exercise.

4. Make the change. In the Streamlit UI, ask these three (or three of your own equivalents):

1. A simple lookup — *"What is the leave policy?"*
2. A multi-part question — *"How does the procurement approval process work and who needs to sign off?"*
3. An edge case — *"What's the policy on bringing pets to the office?"* — likely outside your corpus.

5. Run it. Each question, one at a time, through the UI.

6. What you should see.

- Questions 1 and 2: coherent answers streaming over ~3–6 seconds each.
- Question 3: the LLM probably hallucinates a confident-sounding pet-policy answer. That's not a bug — it's the gap that W6+ RAG work fills. Note it as a known weakness for now.

7. What just happened. By bed-time on Day 1, you have a working FastAPI service, streaming /ask, and a Streamlit UI that calls it. That's the W3 milestone for tonight.

Commit before stopping.

```
git add api/ ui/  
git commit -m "W3 Step 1: FastAPI /ask + /health + Streamlit UI"
```

Step 2 — Mocked unit tests (30 min)

Outcome. Four mocked unit tests running green via pytest in well under a second. No real API calls. The tests catch retry-logic regressions, validation regressions, and accidental changes to ask_llm's call shape.

Step 2 at a glance

Sub-step	Time	What you build
2a	5 min	tests/ + conftest + pytest.ini
2b	10 min	First mocked test + API validation tests
2c	10 min	Retry test (3 hits on 500)
2d	5 min	pytest green — all tests passing

2a — Verify tests/conftest.py + pytest.ini (5 min)

1. What we're doing & why. Set up pytest's discovery + async config. Once in place, every test file we add gets picked up automatically.

2. Where we are now. tests/ exists with __init__.py from 1a. No test files yet, no pytest config.

3. What we're about to change. Two pre-uploaded files: tests/conftest.py (path setup so tests can import from your package root) and pytest.ini (async mode + sensible defaults).

4. Make the change. Both files are in your environment. Verify they're in place:

```
ls tests/conftest.py pytest.ini
```

Open each and read them — both are short. conftest.py adds the project root to sys.path so tests can from src.pipeline.pipeline import ask_llm and from api.main

import app without import gymnastics. `pytest.ini` enables `asyncio_mode = auto` — any `async def test_*` is automatically run with `pytest-asyncio`, no decorator needed on each test.

5. Run it.

```
pytest
```

6. What you should see.

```
===== test session starts
=====
platform linux -- Python 3.11.5, pytest-7.4.3, pluggy-1.3.0
plugins: asyncio-0.21.1
collected 0 items

===== no tests ran in 0.01s
=====
```

Zero tests collected. That's exactly what we want — `pytest` is wired but there's nothing to run yet.

7. What just happened. `Pytest` infrastructure is in place. The next sub-step adds the first actual test.

Watch for. `pytest: error: unrecognized arguments` → the `pytest.ini` has a typo. `pytest --version` to confirm `pytest` itself is healthy.

2b — First mocked test + API validation tests (10 min)

1. What we're doing & why. Two tests, in two files. The first mocks `fake_ask_llm` (the boundary your W2 pipeline calls when `use_fake=True`) and asserts `ask_llm` invokes it correctly. The second uses `FastAPI's TestClient` to confirm `/ask` rejects malformed input and `/health` returns the expected body.

2. Where we are now. `Pytest` runs but collects zero tests.

3. What we're about to change. Create `tests/test_pipeline.py` and `tests/test_api.py`. One test in the first (we'll add the retry test in 2c); two short tests in the second.

4. Make the change.

File 1 — `tests/test_pipeline.py`. Create the file. Write an `async` test function `test_ask_llm_calls_fake_once`. Three-part shape:

- **Arrange.** Build a fake `Answer` with the W2 internal shape — `question`, `text`, `cost_usd`, `retries` fields (import these from `src.pipeline.fake_llm`). Wrap it in `AsyncMock(return_value=fake_answer)`.
- **Act.** Use `patch("src.pipeline.pipeline.fake_ask_llm", AsyncMock(return_value=fake_answer))` as a context manager. **Inside the patched block**, do the import (`from src.pipeline.pipeline import ask_llm`) and call it with a `Question(text="...")`.

- **Assert.** The mock's `call_count == 1` and the returned answer's `.text == "Mocked answer."`.

Mock import order matters. If you import `ask_llm` at the top of the test file (outside the patched block), the import happens *before* the patch is applied — `ask_llm` ends up referencing the unmocked `fake_ask_llm`. Patch first, then import. (This rule applies anywhere in your code, not just tests.)

File 2 — `tests/test_api.py`. Create the file. Write two synchronous tests using `fastapi.testclient.TestClient`:

- `test_ask_rejects_missing_question` — POST `/ask` with empty JSON body, assert status 422 and the error detail mentions the `question` field.
- `test_health_returns_ok` — GET `/health`, assert status 200 and body equals `{"status": "ok"}`.

Completed reference files: `tests/test_pipeline_reference.py` and `tests/test_api_reference.py`. Read them after you've drafted yours, not before.

5. Run it.

```
pytest
```

6. What you should see.

```
collected 3 items

tests/test_api.py::test_ask_rejects_missing_question PASSED
[ 33%] ←
tests/test_api.py::test_health_returns_ok PASSED
[ 66%] ←
tests/test_pipeline.py::test_ask_llm_calls_fake_once PASSED
[100%] ←

===== 3 passed in 0.32s
===== ←
```

Three tests, all green, under half a second total, with zero real LLM calls.

7. What just happened. Your first three tests — two about the API surface, one about the pipeline wrapper. Each tests a deterministic behaviour. None tells you whether the LLM's answers are any *good* — that distinction lands in W5.

Watch for.

- **Wrong patch target.** `patch` operates on the import path *as it appears in the file under test*. W2's `pipeline.py` does `from .fake_llm import fake_ask_llm`, so the function is bound at `src.pipeline.pipeline.fake_ask_llm`. Patch *that* path. Patching `src.pipeline.fake_llm.fake_ask_llm` patches the original but leaves the already-imported reference inside `pipeline.py` alone — the test would still call the real fake.
- **AsyncMock vs MagicMock.** Since `fake_ask_llm` is `async`, the mock must be `AsyncMock` so `await` works on it. A plain `MagicMock` returns a coroutine-shaped

object you can't await.

- **Settings.use_fake = False.** These tests assume the W2 default (use_fake=True). If you've flipped it to False, ask_llm doesn't call fake_ask_llm at all — it goes through the real OpenAI client and the mock is never invoked. Keep the default, or patch the OpenAI client path instead.
-

2c — Retry test (3 hits on 500) (10 min)

1. What we're doing & why. The retry budget from W2 (ask_llm_with_retry) is easy to break by accident — loop count drops to 1, exception type changes, back-off goes infinite. A test that asserts "3 attempts, then raise" guards against all of those.

2. Where we are now. Pytest runs 3 tests, all green. No coverage of retry behaviour yet.

3. What we're about to change. Add a second test to tests/test_pipeline.py.

4. Make the change. Add an async test test_retry_three_times_on_failure. Shape:

- **Arrange.** Two patches in nested with blocks:
 - src.pipeline.pipeline.fake_ask_llm → AsyncMock(side_effect=FakeLLMError("simulated")) so every attempt raises.
 - src.pipeline.pipeline.asyncio.sleep → AsyncMock() so the test doesn't actually wait 1 + 2 = 3 real seconds for the backoff between retries.
- **Act.** Inside the patched block, import ask_llm_with_retry and call it inside with pytest.raises(FakeLLMError):. Pass tries=3 (W2's parameter is tries, not max_retries).
- **Assert.** The fake_ask_llm mock's call_count == 3.

Two assertions in one test. We want both *tried 3 times* (the call count) and *eventually raised* (the pytest.raises). Without pytest.raises, the exception would propagate out of the test and fail it; with pytest.raises, we declare the expected raise as part of the contract.

Why mock asyncio.sleep too. Without that second mock the test honestly waits through the backoff — 1 s + 2 s = 3 s of wall time for one test. That breaks the "all unit tests under a second" budget. Mocking sleep keeps the test instantaneous while still asserting the call-count contract.

Reference: tests/test_pipeline_reference.py.

5. Run it.

```
pytest -v
```

6. What you should see.

```
collected 4 items
```

```
tests/test_api.py::test_ask_rejects_missing_question PASSED  
[ 25%]
```

```
tests/test_api.py::test_health_returns_ok PASSED
[ 50%]
tests/test_pipeline.py::test_ask_llm_calls_fake_once PASSED
[ 75%]
tests/test_pipeline.py::test_retry_three_times_on_failure PASSED
[100%] ←

===== 4 passed in 0.35s
=====
```

7. What just happened. Four green tests guarding four distinct behaviours: client-call shape, retry budget, request validation, health probe. Run pytest after every code change for the rest of the programme.

Watch for.

- **Different retry signature.** If your `ask_llm_with_retry` uses `retries=3` rather than `max_retries=3` or no retry-count parameter at all, adjust the call. The point of the test is to assert the count; the signature is whatever matches your code.
- **Call count of 1 instead of 3.** Your retry loop isn't catching `FakeLLMError`. Check the `except` clause in W2's `ask_llm_with_retry` — it should be `except Exception` (catches anything, including `FakeLLMError`). If it's narrower, either widen it or change the test to raise the type your code catches.

2d — pytest green — final check (5 min)

1. What we're doing & why. A clean final pytest run to confirm nothing regressed while adding tests. Internalise what good output looks like — clean test runs are a habit, not a one-off.

2. Where we are now. 4 tests written.

3. What we're about to change. Nothing.

4. Make the change. No code changes.

5. Run it.

```
pytest --tb=short --durations=5
```

6. What you should see.

```
===== test session starts
=====
collected 4 items

tests/test_api.py::test_ask_rejects_missing_question PASSED
[ 25%]
tests/test_api.py::test_health_returns_ok PASSED
[ 50%]
tests/test_pipeline.py::test_ask_llm_calls_fake_once PASSED
[ 75%]
tests/test_pipeline.py::test_retry_three_times_on_failure PASSED
[100%]

===== slowest 5 durations
=====
←
```

```

0.02s call      tests/test_pipeline.py::test_retry_three_times_on_failure
0.01s call      tests/test_pipeline.py::test_ask_llm_calls_fake_once
0.00s call      tests/test_api.py::test_ask_rejects_missing_question
0.00s call      tests/test_api.py::test_health_returns_ok
===== 4 passed in 0.34s
=====

```

Slowest test is the retry one at ~20 ms (it briefly waits between simulated retries). All four tests done in under half a second. **No real API calls** — that's what mocking buys you.

7. What just happened. Your test suite is green. From now on, run `pytest` after any code change. The point of having tests is being able to refactor with confidence — if the suite stays green, your change is safe.

Commit your work.

```

git add tests/ pytest.ini
git commit -m "W3 Step 2: mocked unit tests + API validation tests"

```

Step 3 — Stress-test the API (45 min)

Outcome. Four stress scenarios run; their behaviours documented in `docs/wk3-stress-notes.md`. You know how your service degrades — not just how it succeeds.

Step 3 at a glance

Sub-step	Time	What you build
3a	10 min	Malformed JSON test
3b	10 min	5000-char question test
3c	10 min	Disconnect mid-stream test
3d	10 min	50 parallel requests
3e	5 min	Document findings

Have `uvicorn` running on port 8000 throughout Step 3.

3a — Malformed JSON (10 min)

1. What we're doing & why. Real callers send bad data. We want to know — without surprises in production — exactly how `/ask` responds to four flavours of bad input.

2. Where we are now. API running. We know `/ask` works on happy-path input. No documented behaviour on broken input.

3. What we're about to change. Hit `/ask` with four bad inputs. Note each response code + body.

4. Make the change. Use your terminal:

```
# Case 1 — empty body
curl -i -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{}'
```

```
# Case 2 — wrong field name (typo)
curl -i -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"q": "What is the leave policy?"}'
```

```
# Case 3 — wrong type (int instead of str)
curl -i -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"question": 42}'
```

```
# Case 4 — malformed JSON (missing closing brace)
curl -i -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"question": "What is the leave policy?"'
```

The `-i` flag tells curl to include response headers, so you can see status codes.

5. Run it. Each curl, one by one.

6. What you should see.

- **Case 1** — HTTP/1.1 422 Unprocessable Entity, body mentions question field is missing.
- **Case 2** — HTTP/1.1 422. Pydantic sees an "extra" field `q` and no required field `question`. Either way the missing `question` triggers 422.
- **Case 3** — HTTP/1.1 422, body says `question` should be a string, got `int`.
- **Case 4** — HTTP/1.1 422 (or 400 depending on FastAPI version). Body mentions a JSON decode error.

7. What just happened. Pydantic handled all four. None reached your code. None cost an LLM call. This is the most underrated benefit of validation — your wallet thanks you.

Watch for. If any came back as a 500, something is wrong: your endpoint is doing manual JSON parsing before Pydantic sees it. Check `api/main.py` — `/ask` should take a `Question` parameter, not a raw `dict`. Let Pydantic do the work.

Keep notes — you'll write them up in 3e.

3b — 5000-character question (10 min)

1. What we're doing & why. Real users sometimes paste in long passages and ask "what does this say". A 5000-character input is well within token limits but enough to noticeably increase latency. We want to know how `/ask` behaves under that load.

2. Where we are now. You've stressed the validation layer. You haven't stressed the LLM layer.

3. What we're about to change. Send one huge question. Time it. Note whether it completes, errors, or hangs.

4. Make the change.

```
# Generate a 5000-char question (a real-ish one repeated)
LONG_Q=$(python -c "print('Please summarize the company policy on remote work. '
* 100)")
echo "Length: ${#LONG_Q}"

# Send it (with -N for streaming and timing)
time curl -N -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d "$(printf '{"question": "%s"}' "$LONG_Q")"
```

5. **Run it.** One curl. It'll take longer than usual.

6. What you should see.

```
Please summarize the company policy on remote work. ...    ← echoes back?
The company allows employees to work remotely up to ...    ← starts streaming
...
real    0m6.821s                                           ← timing
user    0m0.052s
sys     0m0.038s
```

Total time of 5–10 seconds is normal — most of that is the LLM call processing 5000 characters of input.

7. **What just happened.** Long inputs work; latency scales with input length. You now know: 5000-char input → 5–10 second wall time.

Watch for. If it errors with a token-limit error (`context_length_exceeded` or similar), your `ask_llm` doesn't truncate or chunk. That's fine for now — note it as a known limit. Long-context handling comes back in W7.

Keep notes.

3c — Disconnect mid-stream (10 min)

1. **What we're doing & why.** Streams are connections that can die. A user closes their browser tab. A flaky network drops the link. We want to know: does the server clean up its work, or does it leak?

2. **Where we are now.** You know `/ask` succeeds and you know it validates. You don't know what happens when a caller bails.

3. **What we're about to change.** Start a request. Kill it after 1 second. Check the server's logs.

4. **Make the change.** In one terminal — the one running `uvicorn` — keep an eye on the log output. In another:

```
curl --max-time 1 -N -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"question": "Please give me a long answer about something
complicated"}'
```

`--max-time 1` tells curl to give up after 1 second.

5. **Run it.** One curl. It'll be cut short.

6. What you should see.

On the client side:

```
Please give me          ← some words
curl: (28) Operation timed out ← curl bails
```

On the server side (in the uvicorn log), look for:

- A normal `POST /ask` log line
- Possibly a `ClientDisconnect` or similar warning
- **No traceback for an unhandled exception** ← the key thing

7. What just happened. The server kept generating tokens until it tried to write the next chunk to a connection that was already closed. At that point it gets a `ClientDisconnect` from FastAPI/Starlette, which is handled cleanly. The async generator is garbage-collected.

If you saw a long traceback in the server log, your stream generator has a bug — usually catching `Exception` somewhere and not re-raising or cleaning up.

Watch for. What matters here isn't the status code — the connection was closed before any status code is meaningful. What matters is whether the server is healthy after. Hit `/health` again:

```
curl http://localhost:8000/health
```

You should still get `{"status": "ok"}`. If the server is locked up, something needs fixing.

Keep notes.

3d — 50 parallel requests (10 min)

1. What we're doing & why. Real services get hit by more than one user at a time. We want to know how `/ask` behaves at modest concurrency: 50 total requests, up to 10 in flight at any moment. Any errors? What's the p95 latency? Does SQLite capture all of them?

2. Where we are now. You've tested validation, big inputs, and disconnects — all single-request.

3. What we're about to change. Run the pre-uploaded stress script. It spawns 50 concurrent calls and reports stats.

4. Make the change. The script is in your environment at `scripts/stress_test.py`. Run it:

```
python scripts/stress_test.py --requests 50 --concurrent 10
```

Internally it uses `httpx.AsyncClient` to fire 50 POST requests with a semaphore capping concurrent in-flight calls at 10. Open the script if you want to see how — but you don't need to edit it.

5. Run it. One python command. It'll take a minute or two.

6. What you should see.

```
Stress test: 50 requests, up to 10 concurrent
```

```

Total wall time: 72.14s
Successes:      50 / 50
Effective req/s: 0.69

Latency (successful requests):
  min: 5.12s
  p50: 6.81s
  p95: 9.42s
  max: 10.71s

```

← worst-case

Exact numbers vary with your network and OpenAI's load.

Verify SQLite captured all 50:

```

sqlite3 results.db "SELECT COUNT(*) FROM answers WHERE created_at >
datetime('now', '-5 minutes');"
# Expected: 50

```

7. What just happened. Your service handles ~10 concurrent users without errors at roughly 0.7 requests/second. The bottleneck is the LLM call itself — your code added ~50 ms per request; the LLM added ~5 seconds. p95 under 10 seconds for a streamed answer is fine for an internal tool. Throughput optimisation comes back in W27.

Watch for.

- **OpenAI rate limit.** Free-tier accounts cap concurrent calls. The script will report 429s. Drop `--concurrent` from 10 to 3 and re-run.
- **Local file descriptor limit.** Less common, but `ulimit -n 1024` is the default on small VMs. `ulimit -n 4096` solves it.
- **ReadTimeout.** Bump the timeout in `scripts/stress_test.py` from 60 to 120 if your network is slow.
- **SQLite count mismatch.** Fewer than 50 entries → your W2 persistence is dropping writes under concurrency. Note it; fix lands in W6.

Keep notes.

3e — Document findings in `docs/wk3-stress-notes.md` (5 min)

1. What we're doing & why. Findings die unwritten. A 5-minute note now becomes ammo for W4, W6, W27 — and for DR #1 in W5 where you'll be asked *"what do you know about your service's limits?"*

2. Where we are now. You've run four stress scenarios and have informal notes.

3. What we're about to change. Create `docs/wk3-stress-notes.md` with four short findings, one section each.

4. Make the change. Create `docs/wk3-stress-notes.md`. Cover these four sections at minimum (one paragraph each is enough):

- **Finding 1 — Malformed JSON.** Response codes for each of the four bad inputs; whether any reached your application code.

- **Finding 2 — 5000-character question.** Total wall time observed; whether the API hit a token limit; whether streaming worked.
- **Finding 3 — Disconnect mid-stream.** What the server logged on `--max-time 1`; whether `/health` still responded after.
- **Finding 4 — 50 parallel requests.** Successes out of 50; p50 / p95 latency; effective req/s; whether SQLite captured all entries.

Add a short "**Known limits / follow-ups**" section at the end listing anything to fix later (token-limit handling, SQLite write contention, cost tracking).

5. Run it.

```
git add docs/wk3-stress-notes.md
```

6. What you should see. The file shows up in `git status` as a new file.

7. What just happened. You have a written record of what your service does under non-happy-path conditions. This file lives in your repo and gets cited in your DR #1 presentation in W5.

Commit your work.

```
git commit -m "W3 Step 3: stress-test findings"
```

Step 4 — Update ADR with the API contract (15 min)

Outcome. `docs/adr/0002-api-contract.md` exists, filled in for your capstone, and committed. The `/v1/ask` contract is locked.

Step 4 at a glance

Sub-step	Time	What you build
4a	3 min	Drop the ADR template into place
4b	10 min	Fill in the v1 contract
4c	2 min	Commit

4a — Drop the ADR template into place (3 min)

1. What we're doing & why. The contract belongs in its own ADR (0002), not buried inside 0001. We're capturing the API surface as a separate decision so it can be referenced and updated independently.

2. Where we are now. `docs/adr/0001-capstone-framing.md` exists from W1. No 0002.

3. What we're about to change. Copy the pre-uploaded template into place.

4. Make the change. The template is in your environment at `docs/adr/0002-api-`

`contract-template.md`. Copy it to the working file:

```
cp docs/adr/0002-api-contract-template.md docs/adr/0002-api-contract.md
```

Open it in your editor. Skim the structure — Context, Decision (with endpoints), Versioning Rule, Consequences, Tests.

5. Run it.

```
ls docs/adr/
```

6. What you should see.

```
0001-capstone-framing.md
0002-api-contract.md ←
```

7. What just happened. The ADR file is in place with a template structure. The next sub-step fills in your specifics.

4b — Fill in the v1 contract (10 min)

1. What we're doing & why. The template has the structure but generic content. Make it yours: your use case in the Context, your capstone-specific fields in the Decision section, your test file names in the Tests section.

2. Where we are now. Template-filled ADR.

3. What we're about to change. Five edits:

- Header.** Replace `_[your name]_` and `_[YYYY-MM-DD]_`.
- Context.** Add 1–2 sentences naming *your* capstone use case (e.g. "Enterprise Knowledge Assistant for HR policies", "Customer Support Triage Bot") so future readers know what API this is for.
- Request body table.** If your `Question` model has extra fields, add them. For most learners it's just `question: str`.
- Response body table.** List every field on your `Answer`.
- Tests securing this contract.** Update to match your actual test file paths from Step 2.

4. Make the change. Open `docs/adr/0002-api-contract.md` and edit. The endpoint tables, versioning rule, and tests sections can stay close to the template unless your code differs.

5. Run it. Re-read the whole file end-to-end. Imagine you're a peer reviewer at DR #1 — does it make sense? Does it match what your `api/main.py` actually does?

If something is off, fix it. The ADR is the source of truth; if it disagrees with the code, the code is wrong (or the ADR needs an update).

6. What you should see. A complete ADR with your specifics filled in — no template placeholders left.

7. What just happened. You have a locked API contract written down in language anyone can

read. W4's internal upgrades will not change a word of this file.

4c — Commit (2 min)

1. What we're doing & why. Lock the work in. The git history is part of the M1 deliverable evidence — DR #1 reviewers will look at it.

2. Where we are now. ADR filled in but not committed.

3. What we're about to change. One commit.

4. Make the change.

```
git add docs/adr/0002-api-contract.md
git commit -m "W3 Step 4: ADR 0002 — /v1/ask contract locked"
```

5. Run it.

```
git log --oneline -5
```

6. What you should see.

```
a3f9c2b W3 Step 4: ADR 0002 — /v1/ask contract locked ←
b1d4e8a W3 Step 3: stress-test findings
c5a2f3d W3 Step 2: mocked unit tests + API validation tests
d8e9c1b W3 Step 1: FastAPI /ask + /health + Streamlit UI
... earlier W2 commits ...
```

Four W3 commits, in order. **Week 3 is done.**

End-of-week summary

By the end of Week 3 your capstone repo has:

- ☒ **api/main.py** — FastAPI service with /ask (streaming), /ask_batched (non-streaming), /health.
- ☒ **ui/app.py** — Streamlit UI hitting /ask.
- ☒ **tests/test_pipeline.py** — 2 mocked unit tests (fake call + retry).
- ☒ **tests/test_api.py** — 2 API surface tests (validation + health).
- ☒ **tests/conftest.py + pytest.ini** — pytest config.
- ☒ **scripts/stress_test.py** — 50-parallel stress harness.
- ☒ **docs/wk3-stress-notes.md** — 4 stress findings documented.
- ☒ **docs/adr/0002-api-contract.md** — /v1/ask contract locked.
- ☒ **Git history** with at least 4 W3 commits.

Single check before declaring done:

```
# In one terminal
```

```
uvicorn api.main:app --port 8000

# In another
streamlit run ui/app.py --server.port 8501
pytest                                     # all green
python scripts/stress_test.py --requests 10 --concurrent 3      # quick
replay
```

If all four pieces respond as expected, Week 3 is shipped.

What changes next week (W4)

W4 opens up the body of `/ask` — token-aware cost tracking, three structured-output approaches (Pydantic + JSON mode + tool-calling), real LLM-side streaming, and the model landscape (GPT / Claude / Ollama).

The `/v1/ask` contract you locked in `0002-api-contract.md` will not change. That's the whole point.

When you start W4, the same curl command you wrote in 1f should still work — but the response body underneath will be richer (real cost from `response.usage`, optional `sources`, optional `confidence`, `schema_version: "v1"`).

Troubleshooting reference

ModuleNotFoundError: No module named 'pipeline'

You're running uvicorn or pytest from inside `api/` or `tests/`. Run from the repo root.

Streaming endpoint dumps everything at once instead of streaming

Two possible causes:

1. Your function uses `return` instead of `yield` — it's not a generator.
2. You're hitting it with curl without the `-N` flag. The server is streaming; curl is buffering. Add `-N`.

pytest collects 0 items

Your `pytest.ini` might be missing or your test files might not start with `test_`. Pytest defaults: files match `test_*.py`, functions match `test_*`.

AsyncMock returns coroutine instead of value

You set `return_value=fake_response` on a regular `MagicMock` instead of `AsyncMock`. Use `AsyncMock(return_value=...)` so that `await` works on it.

Streamlit shows "Cannot reach the API"

The API isn't running, or it's on a different port than the UI expects. Check the uvicorn terminal. Confirm `API_URL` in `ui/app.py` matches the port uvicorn is listening on.

422 errors on requests that look correct

Probably a content-type mismatch. Make sure your curl includes `-H "Content-Type: application/json"`. From a UI, it's automatic.

OpenAI rate limit hit during stress test

Free-tier OpenAI accounts have low concurrent-call caps. Reduce `--concurrent` to 3:

```
python scripts/stress_test.py --requests 50 --concurrent 3
```

unittest.mock.patch not patching the right thing

`patch` operates on the import path *as it appears in the file under test*. Your `W2 pipeline.py` does from `.fake_llm import ...`, so `fake_ask_llm` is bound inside that module at `src.pipeline.pipeline.fake_ask_llm`. Patch *that* path, not `src.pipeline.fake_llm.fake_ask_llm` (the original definition).

pytest-asyncio complains about missing event loop

You're probably missing `asyncio_mode = auto` in `pytest.ini`. Or you've declared an `async def test_*` but `pytest-asyncio` isn't installed. `pip list | grep asyncio` to check.

End of Week 3 lab guide.